

DOCUMENTO TÉCNICO DE DISEÑO

Julian David Miranda Leguizamón

Sebastian Bedoya Florez

Edgar Andres Romero Otalora

Jhon Alexander Tenjo Romero

Universidad de la Sabana

Maestría en Arquitectura de Software

Miguel Alfonso Varela Fonseca

Bogotá D.C.

25/05/2026

Índice

RESUMEN	4
ANÁLISIS EXPLORATORIO DE DATOS (EDA) DEL DATASET BRAZILIAN E-COMMERCE	5
ANÁLISIS DE REQUISITOS	7
Contexto de Ecommify	7
Requisitos no funcionales	8
Identificación de entidades del dominio	8
Tabla de volúmenes de datos esperados	9
DISEÑO CONCEPTUAL	10
Relaciones	10
Restricciones de negocio	11
Pedidos	11
Productos de los pedidos	11
Valor de los productos	11
Cuotas de pago	11
Registro de reseñas	11
Ubicaciones únicas	11
DISEÑO LÓGICO - MÓDULO POSTGRESQL	12
Esquema relacional normalizado	12
Tipos de datos utilizados	12
UUID	12
Varchar	12
Int2 (Small int)	12
Int8 (Integer)	12
Numeric	13
Jsonb	13
Timestamp	13
Extensiones	13
UUID-OSSP	13
PG_TRGM	13
DISEÑO LÓGICO PRELIMINAR - MÓDULO MONGODB	14
Esquema de documentos	14
Products	14
Order reviews	14
Geolocation	14
Patrones de Modelado	15
Patrón de Atributos (products)	15
Patrón de Cómputo (products)	15
Patrón Polimórfico (order_reviews)	15
Justificación de Distribución de Datos	15

PostgreSQL como el "Write Model"	16
MongoDB como el "Read Model"	16
DECISIONES ARQUITECTÓNICAS	17
Patrón transaccional compuesto	17
Matriz de Decisiones de Almacenamiento SQL (PostgreSQL)	18
Matriz de Decisiones de Almacenamiento NoSQL (MongoDB)	19
ANÁLISIS DEL TEOREMA CAP	20
Sacrificio de la Consistencia Estricta (C)	20
Resiliencia ante Particiones de Red (T)	20
Trade-offs y estrategias de mitigación	20
Pérdida de Consistencia Inmediata	21
Duplicidad de datos y mayor consumo de almacenamiento	21
Flujos de Sincronización	22
Sincronización Transaccional a Consulta (Ida: PostgreSQL → MongoDB)	22
Sincronización de Interacción a Analítica (Vuelta: MongoDB → PostgreSQL):	22

RESUMEN

Este documento presenta el diseño conceptual, lógico y arquitectónico de una plataforma de comercio electrónico basada en una Arquitectura Políglota Híbrida, combinando PostgreSQL y MongoDB para aprovechar las fortalezas de ambos paradigmas de almacenamiento. La solución fue diseñada a partir del análisis de los datasets de Olist, considerando volumen de datos, relaciones transaccionales, patrones de consulta y requerimientos de escalabilidad. PostgreSQL se seleccionó como motor principal para el procesamiento transaccional crítico, garantizando integridad referencial, consistencia ACID y control estructurado sobre entidades como órdenes, pagos, clientes y vendedores. Por otra parte, MongoDB fue elegido para manejar datos semiestructurados y consultas de alta concurrencia relacionadas con catálogo de productos, reseñas y geolocalización, permitiendo esquemas flexibles y acceso optimizado mediante documentos desnormalizados. La arquitectura implementa sincronización asíncrona orientada a eventos mediante Apache Kafka para mantener consistencia eventual entre ambos motores sin afectar el rendimiento operacional. Adicionalmente, se aplicaron patrones de modelado documental, análisis CAP y estrategias de mitigación para disponibilidad y tolerancia a fallos. El resultado es una solución escalable, flexible y preparada para soportar cargas analíticas y transaccionales simultáneamente.

ANÁLISIS EXPLORATORIO DE DATOS (EDA) DEL DATASET BRAZILIAN E-COMMERCE

Como el dataset está implementado y distribuido en tablas, se debe realizar una desnormalización para obtener una tabla **OrdersFull** y a partir de ahí, hacer el análisis exploratorio de datos (EDA). Este análisis contiene un análisis de duplicados, análisis de distribuciones, análisis de relaciones entre entidades, análisis de valores nulos, análisis de outliers y un análisis de patrones temporales y geográficos.

La tabla **OrdersFull**, es una tabla de 33 columnas donde se mezclaron las tablas **olist_orders_dataset** siendo esta la tabla principal y donde `order_id` tendrían valores únicos, `olist_customers_dataset`, `olist_order_items_dataset`, `olist_products_dataset`, `olist_sellers_dataset`, `olist_order_payments_dataset`, `olist_order_reviews_dataset` y `product_category_name_translation`.

En la desnormalización, en la tabla pagos se hizo una agrupación para que un pedido tuviera solo un pago, y en la tabla reviews también se realizó una agrupación para que un pedido solo tuviera una calificación.

En el análisis de duplicados no hay registros duplicados en la tabla en general, los pedidos tienen el 12.33% de duplicación debido a que un pedido puede tener varios ítems y la cantidad de productos duplicados es alta (70.95%) ya que un producto puede estar en varios pedidos.

Se hizo un análisis de distribución de las características de un producto donde predominan los productos pequeños o livianos.

Y en la distribución del dinero, se tomaron las columnas relacionadas al precio, pagos o valor del pedido, y también se evidencia que la frecuencia se da en los precios bajos.

En el análisis de relación entre entidades el promedio de productos en un pedido es de 3.11, un vendedor en promedio vende 32.31 pedidos, y arrojó 1 pedido por cliente, debido a la desnormalización ya que se quería que no hubieran dos pedidos con el mismo id.

El porcentaje de valores nulos en cada columna está en el rango de 0 a 2.85%. Las columnas con mayor porcentaje de valores nulos son las relacionadas a las características del producto.

Mediante el análisis de detección de anomalías, se validó si hay registros en las columnas numéricas que tengan valores menores o iguales a 0, que la columna de calificación esté entre 1 y 5, que las fechas no sean mayores a la fecha actual y que haya coherencia entre fechas. El único inconveniente que arrojó fue 187 registros, tienen fecha de envío antes de la fecha de compra.

El análisis de outliers en las columnas de precios arrojó que los pagos se concentran entre 0 y 500, los precios de los productos se concentran entre 0 a 300 y el valor de los pedidos se concentra entre 0 a 40.

Y para terminar, con el análisis de patrones temporales y geográficos indica que Sao Paulo es la ciudad con más clientes con más de 14000 y le sigue Río de Janeiro con aproximadamente la mitad de los clientes de Sao Paulo. El costo de envío por estado, el estado más costoso está aproximadamente en 35 y el menos costoso en 12. En enero de 2017 comenzó a aumentar la cantidad de pedidos por mes, en octubre se mantuvo y comenzó a disminuir en agosto de 2018. Los lunes y martes son los días que más hacen pedidos y los sábados son los días que menos hacen pedidos.

ANÁLISIS DE REQUISITOS

Contexto de Ecommify

Ecommify es una empresa de comercio electrónico enfocada en conectar vendedores y compradores dentro de una plataforma digital centralizada. La solución permitirá a diferentes comercios ofrecer productos en línea, gestionar órdenes de compra, procesar pagos y coordinar entregas a clientes ubicados en distintas regiones.

La plataforma administra múltiples procesos del negocio, incluyendo catálogo de productos, gestión de clientes, pagos electrónicos, seguimiento de pedidos, logística de distribución y calificaciones de experiencia de compra. Además, incorporará funcionalidades de análisis geográfico para identificar patrones de ventas, comportamiento de clientes y distribución de vendedores según ubicación.

Debido al crecimiento esperado del marketplace y al alto volumen de transacciones y consultas analíticas, Ecommify implementará una arquitectura escalable basada en una estrategia Políglota Híbrida, permitiendo combinar tecnologías orientadas al procesamiento transaccional y analítico. Esto facilitará mantener un alto rendimiento en operaciones de compra en tiempo real y, al mismo tiempo, soportar generación de reportes, dashboards y análisis de negocio para la toma de decisiones estratégicas.

Requisitos funcionales

1. Gestionar clientes y su información geográfica.
2. Registrar y administrar órdenes de compra.
3. Gestionar productos y categorías.
4. Administrar vendedores y marketplaces asociados.
5. Registrar pagos y métodos de pago.
6. Gestionar ítems asociados a cada orden.
7. Registrar reseñas y calificaciones de clientes.

8. Permitir consultas analíticas sobre ventas, entregas y satisfacción.
9. Generar reportes históricos y dashboards.
10. Relacionar información geográfica mediante códigos postales y coordenadas.

Requisitos no funcionales

Categoría	Requisito
Rendimiento	Soportar consultas analíticas sobre más de 1 millón de registros geográficos
Escalabilidad	Permitir crecimiento horizontal de datos y servicios
Integridad	Mantener consistencia entre órdenes, pagos y productos
Mantenibilidad	Arquitectura modular basada en microservicios y BD híbrida
Analítica	Optimizar operaciones OLAP y generación de reportes
Compatibilidad	Integración con PostgreSQL y motores NoSQL
Disponibilidad	Garantizar acceso continuo a órdenes y pagos

Identificación de entidades del dominio

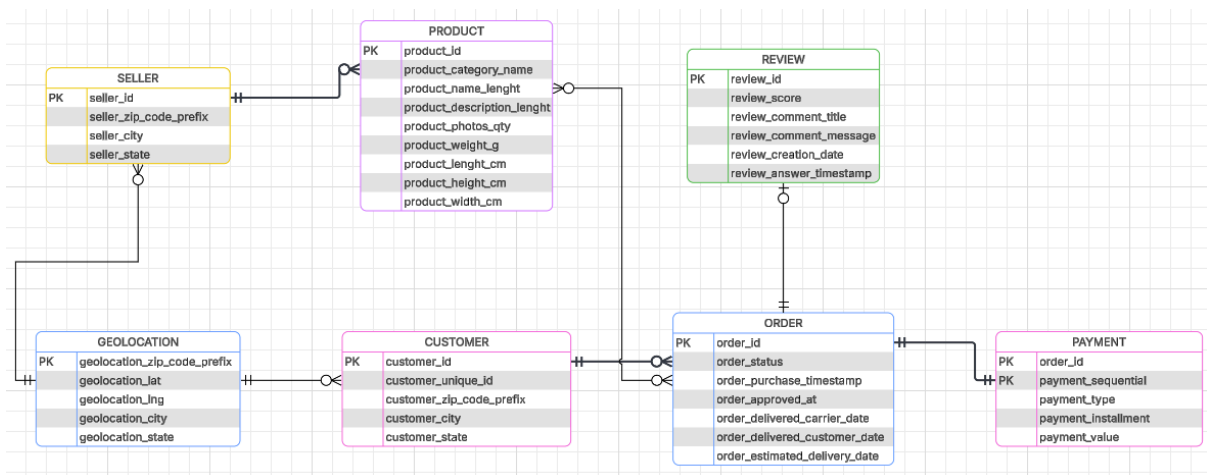
Entidad	Descripción
Customer	Información de clientes
Orders	Órdenes realizadas en la plataforma
Order_Items	Productos asociados a cada orden
Payments	Información de pagos
Reviews	Calificaciones y comentarios
Products	Catálogo de productos
Sellers	Información de vendedores

Geolocation	Información geográfica
Categories	Traducción de categorías

Tabla de volúmenes de datos esperados

Tabla	Cantidad aproximada de registros	Crecimiento esperado
customers	99,441	Alto
orders	99,441	Alto
order_items	112,650	Muy alto
order_payments	103,886	Alto
order_reviews	99,224	Medio
products	32,951	Medio
sellers	3,095	Bajo
geolocation	1,000,163	Muy alto
categories	71	Bajo

DISEÑO CONCEPTUAL



Relaciones

1. Customer → Order (1:N): Un cliente puede realizar muchos pedidos y cada pedido pertenece solo a un cliente.
2. Order → Product (N:N): Un pedido puede llevar muchos productos y los productos pueden ser comprados en varios pedidos.
3. Order → Review (1:N): Un pedido puede recibir cero o varias reseñas.
4. Order → Payment (1:N): Un pedido puede ser pagado mediante uno o más pagos parciales o combinados. Cada transacción de pago está ligada a una sola orden.
5. Seller → Product (1:N): Un vendedor puede vender / ofertar varios productos.
6. Geolocation → Customer / Seller

Restricciones de negocio

Pedidos

Un pedido pertenece a un único cliente. Un cliente puede realizar muchas órdenes a lo largo del tiempo. No puede existir un pedido si no está asociado a un cliente y a un vendedor.

Productos de los pedidos

Una orden debe tener al menos 1 producto para ser válida. La cantidad de productos adquiridos en un pedido debe ser un entero mayor o igual a 1.

Valor de los productos

Cuando un cliente compra un producto, el precio debe quedar fijo en su compra, ya que si el vendedor / tienda decide bajar o subir el precio del producto en el catálogo, el valor de las compras ya realizadas no se pueden alterar.

Cuotas de pago

El número de cuotas seleccionadas por ejemplo con una tarjeta de crédito debe ser como mínimo 1. No existen las compras con cuotas negativas.

Registro de reseñas

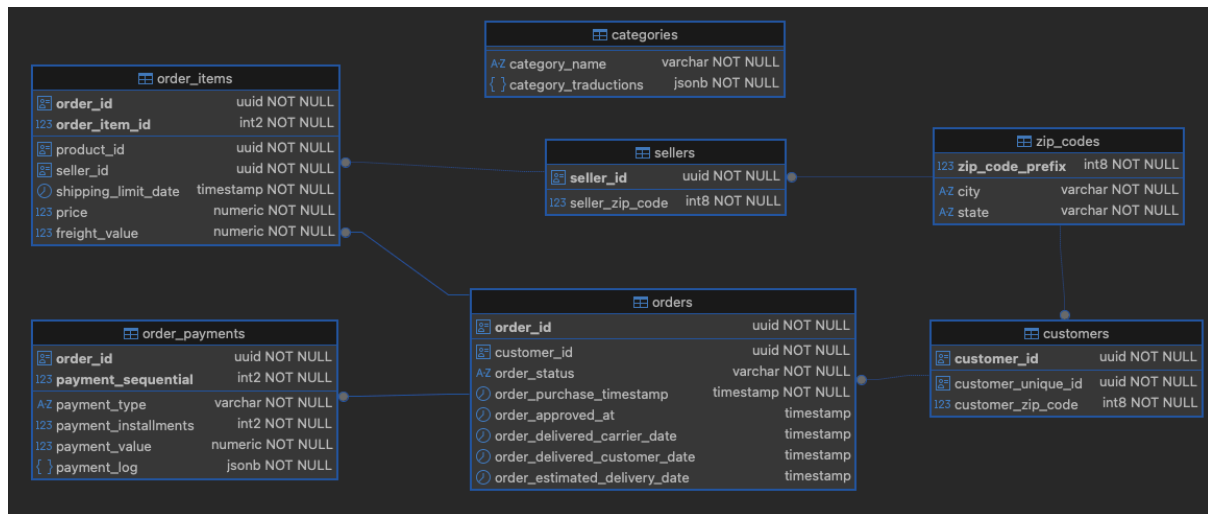
Un cliente solo puede dejar una reseña si efectivamente realizó la compra de un pedido.

Ubicaciones únicas

El sistema no permite registrar coordenadas iguales (misma latitud y longitud) repetidas bajo un mismo código postal.

DISEÑO LÓGICO - MÓDULO POSTGRESQL

Esquema relacional normalizado



Tipos de datos utilizados

UUID

Se utiliza para identificadores únicos ya que garantiza unicidad global, evita cruces entre registros e incrementa la seguridad al no utilizar IDs secuenciales. Ejemplo: las llaves foráneas de las tablas ordenes, clientes, vendedores, entre otros.

Varchar

Se utiliza para almacenar cadenas de texto de longitud variable, permite definir límites máximos de longitud y es adecuado para datos textuales controlados. Ejemplo: estados de las órdenes, tipos de pagos, entre otros.

Int2 (Small int)

Se utiliza para almacenar valores pequeños y acotados como por ejemplo: número de cuotas, secuencia del pago, entre otros.

Int8 (Integer)

Se utiliza para almacenar valores enteros de mayor rango como por ejemplo el prefijo de los códigos postales.

Numeric

Se utiliza para datos de valores decimales ya que es fundamental para cálculos monetarios como por ejemplo: valor del pedido, precio de los productos, entre otros.

Jsonb

Se utiliza para almacenar información semiestructurada en formato json ya que permite realizar consultas eficientes. Ejemplo: campo para almacenar las traducciones de las categorías de los productos y el log de la transacción del pago del pedido.

Timestamp

Se utiliza para almacenar fecha y hora, como por ejemplo las fechas y horas de las diferentes etapas de la compra de un pedido hasta su entrega al cliente.

Extensiones

UUID-OSSP

Para usar el tipo de dato UUID y permitir que la base de datos genere automáticamente los id por si misma, es necesario activar la extensión uuid-oss. La extensión le permite a PostgreSQL tener un conjunto de funciones avanzadas, por ejemplo: `uuid_generate_v4()`. Esto permitirá que cuando se inserten registros en las tablas orders, customers, sellers, entre otros, se generará automáticamente un id único sin depender de que un lenguaje de programación lo deba realizar.

PG_TRGM

La implementación de índices GIN en las columnas city (zip_codes) y category_name (categories) hace necesario el uso de pg_trgm. La extensión divide las cadenas de texto en bloques de tres caracteres consecutivos (trigramas) para medir la similitud matemática entre palabras. Esto permitirá que el sistema sea tolerante a errores ortográficos que puedan ser cometidos por los usuarios al momento de digitar su ciudad al momento de realizar sus pedidos, también permitirá optimizar las búsquedas de las categorías de los productos y de

esta forma el sistema podrá ofrecer sugerencias automáticas al usuario cuando realiza búsquedas.

DISEÑO LÓGICO PRELIMINAR - MÓDULO MONGODB

Esquema de documentos

Products

```
{
  "_id": "060cb19345d90064d1015407193c6ffc",
  "product_category_name": "perfumeria",
  "physical_specs": {
    "weight_g": 525,
    "length_cm": 17,
    "height_cm": 11,
    "width_cm": 12
  },
  "multimedia_metadata": {
    "product_name_length": 42,
    "product_description_length": 774,
    "photos_qty": 2
  },
  "performance_metrics": {
    "average_review_score": 4.6,
    "total_reviews_count": 18
  }
}
```

Order reviews

```
{
  "_id": "7ac549e1e29c625d3934329e43b546e5",
  "order_id": "e481f51cbdc54678b7349021f2804312",
  "review_score": 5,
  "review_comment": {
    "title": "Excelente servicio",
    "message": "El producto llegó mucho antes de lo estimado y en perfectas condiciones."
  },
  "timestamps": {
    "creation_date": "2017-09-21T00:00:00Z",
    "answer_timestamp": "2017-09-22T10:57:03Z"
  }
}
```

Geolocation

```
{
  "_id": "01001",
  "geolocation_zip_code_prefix": 1001,
  "location": {
    "type": "Point",
    "coordinates": [-46.634007, -23.549807]
  },
  "city": "sao paulo",
  "state": "SP"
}
```

Patrones de Modelado

Patrón de Atributos (products)

En lugar de tener un esquema rígido con cientos de campos vacíos, las propiedades físicas y métricas se agrupan en subobjetos indexables (`physical_specs`). Esto permite añadir características variables en el futuro sin alterar la estructura base de la colección.

Patrón de Cómputo (products)

Calcular la calificación promedio ejecutando operaciones AVG y COUNT sobre millones de reseñas en tiempo real destruiría el rendimiento del catálogo. Mediante este patrón, el documento del producto ya almacena los valores calculados `average_review_score` y `total_reviews_count`. Estos datos se pre calculan y actualizan en segundo plano sólo cuando se genera una nueva reseña.

Patrón Polimórfico (order_reviews)

Las interacciones de los usuarios varían ya que algunos solo asignan puntuación y otros escriben textos largos. El documento se adapta polimórficamente, almacenando el subobjeto de comentarios únicamente si el usuario aportó texto, optimizando drásticamente el espacio en disco y la eficiencia de los índices.

Justificación de Distribución de Datos

La distribución estratégica de los datos responde estrictamente al diseño de la Arquitectura Híbrida General del sistema, la cual implementa una segregación de responsabilidades de lectura y escritura:

PostgreSQL como el "Write Model"

De acuerdo con el flujo arquitectónico propuesto, aquí se procesan y almacenan de forma exclusiva las operaciones transaccionales críticas del negocio que exigen mutación de estado e integridad absoluta: la creación de órdenes (orders), la confirmación y auditoría de pasarelas de pago (order_payments), y el control maestro de entidades reguladas (customers y sellers). Al ser la fuente de comandos de escritura, este motor garantiza la consistencia estricta bajo propiedades ACID, procesando esquemas avanzados como estructuras JSONB y tipos compuestos para obtener la metadata de la compra en el mismo instante en que se ejecuta la transacción.

MongoDB como el "Read Model"

Su propósito exclusivo es servir de manera optimizada el tráfico masivo de consultas concurrentes de cara al usuario final en la interfaz (navegación del catálogo de productos, visualización de opiniones de compradores e indexación de prefijos logísticos mediante coordenadas GeoJSON). Al deslocalizar estas entidades documentales hacia MongoDB, se aísla por completo el tráfico pesado de búsquedas del core transaccional de pagos. Esto permite que el modelo opere bajo esquemas desnormalizados de alta velocidad y disponibilidad, sin penalizar el rendimiento ni competir por los recursos de hardware del Write Model.

DECISIONES ARQUITECTÓNICAS

Patrones transaccionales

Patrón transaccional compuesto

En la operación de la compra de un pedido un cliente efectúa un pago, el sistema ejecuta un patrón transaccional que involucra la escritura simultánea en tres entidades: orders (creación del registro), order_items (detalle de productos del pedido) y order_payments (detalle del pago del pedido), si alguna de las inserciones falla toda la operación debe revertirse para evitar inconsistencias de datos.

Patrón de alta lectura

Las consultas del catálogo de productos y las reseñas de los usuarios presentan una tasa alta de lectura por parte de los clientes en la plataforma, por el contrario se presenta una escritura baja ya que los productos se actualizan ocasionalmente y las reseñas son creadas si se efectúa una compra.

Patrón de mutación secuencial

Los pedidos tienen un ciclo de vida logístico por dicha razón la entidad orders tiene un patrón de actualización continua y lineal basado en el estado del pedido como por ejemplo: aprobación del pago, despacho de la transportadora, entrega al cliente

Patrón volumétrico

El sistema debe calcular el costo de envío, para esto el sistema requiere realizar una consulta combinada tomando el código postal del cliente y el del vendedor para buscar sus coordenadas en la colección geolocation para calcular la distancia.

Matriz de Decisiones de Almacenamien

to SQL (PostgreSQL)

Colección	Estructura	Volumen y Velocidad	Teorema CAP
customers	Estructura altamente relacional. Contiene identificadores únicos de clientes, ciudad, estado y código postal. Relacionada directamente con orders.	Volumen medio (~99k registros). Velocidad media de lectura y escritura debido a consultas frecuentes de clientes y ubicación.	(C) PostgreSQL prioriza consistencia para garantizar integridad de datos de clientes y relaciones con pedidos.
orders	Tabla central transaccional con múltiples relaciones hacia customers, payments y order_items. Maneja estados y fechas del ciclo de vida del pedido.	Alto volumen (~99k pedidos) y alta velocidad transaccional. Escrituras frecuentes y consultas constantes.	(C) La consistencia es crítica para evitar pérdida o duplicidad de pedidos durante transacciones.
order_items	Estructura relacional detallada por producto vendido. Relaciona órdenes, productos y vendedores.	Alto volumen (~112k registros). Alta velocidad de inserción y consultas analíticas sobre ventas.	(C) Debe garantizar integridad entre pedidos, productos y vendedores.
order_payments	Datos financieros estructurados asociados a órdenes. Incluye cuotas, tipo de pago y montos.	Volumen alto (~103k registros). Escrituras moderadas y lecturas frecuentes para análisis financiero.	(C) Se prioriza consistencia para asegurar exactitud de pagos y facturación.
sellers	Información estructurada de vendedores con ubicación geográfica. Relacionada con order_items.	Volumen bajo (~3k vendedores). Lecturas frecuentes y pocas actualizaciones.	(C) Se requiere consistencia para mantener referencias correctas en ventas y logística.
categories	Tabla pequeña de traducción de	Volumen muy bajo (71 registros). Baja	(CA) Puede mantenerse

Colección	Estructura	Volumen y Velocidad	Teorema CAP
	categorías entre portugués e inglés. Relación auxiliar con products.	frecuencia de acceso.	altamente disponible y consistente debido a su tamaño reducido y baja complejidad.

Matriz de Decisiones de Almacenamiento NoSQL (MongoDB)

Colección	Estructura	Volumen y Velocidad	Teorema CAP
products	Estructura polimórfica: Los productos varían sus atributos según su categoría (ej: ropa utiliza tallas).	Volumen: Medio (~32k registros). Velocidad: Extremadamente Alta (Lecturas masivas concurrentes).	(P) El sistema está diseñado para sobrevivir a una división del clúster. (A) Ante dicha partición, el catálogo debe seguir respondiendo y mostrando los productos.
order_reviews	Estructura semi-estructurada: Documentos independientes vinculados por product_id. No todas las reseñas tienen la misma estructura.	Volumen: Alto (+100k registros y crece con cada orden). Velocidad: Media en Escritura y Alta en Lectura.	(P) Si la red se fragmenta entre centros de datos, los servidores de base de datos aislados siguen operando de forma autónoma. (A) Un cliente debe poder escribir o leer comentarios siempre. La caída de un enlace de red no puede bloquear la sección de opiniones.
geolocation	Estructura documental especializada (GeoJSON): Almacena puntos	Volumen: Alto (Más de 1 millón de filas en el dataset original).	(P) MongoDB maneja esta carga mediante Sharding. Si un nodo fragmentado

	espaciales definidos por arreglos de coordenadas [longitud, latitud].	Velocidad: Alta (Cálculos de proximidad en milisegundos para estimación de costo de envío durante el checkout).	regional sufre una desconexión, los demás shards siguen atendiendo consultas de sus respectivas regiones. (A) El cálculo del costo de envío en el carrito debe responder en milisegundos para evitar que el usuario abandone la compra.
--	---	---	--

ANÁLISIS DEL TEOREMA CAP

Sacrificio de la Consistencia Estricta (C)

En las colecciones de products y order_reviews forzar una consistencia lineal exigiría que, ante una actualización de precio o creación de un comentario, todas las replicas se bloqueen hasta confirmar el cambio. Esto degradará la velocidad de la plataforma drásticamente.

Se decide optar por un modelo de consistencia eventual sincronizando las replicas de forma asíncrona a través del Message Broker.

Resiliencia ante Particiones de Red (T)

En un entorno distribuido en la nube los fallos son inevitables, cuando alguna comunicación entre particiones se interrumpe, MongoDB prioriza mantenerse operativo y responder a lecturas desde nodos secundarios guardando y protegiendo así la disponibilidad.

Trade-offs y estrategias de mitigación

Al adoptar un enfoque arquitectónico AP en MongoDB para optimizar las consultas masivas de la plataforma, se asume y mitiga los siguientes trade-offs:

Pérdida de Consistencia Inmediata

Riesgo

Al priorizar la disponibilidad masiva (A), si un administrador actualiza el nombre o la categoría de un producto en el Write Model (PostgreSQL), un cliente que navegue por el catálogo NoSQL en ese mismo instante podría visualizar los datos anteriores durante una ventana de milisegundos mientras se propaga el cambio.

Estrategia de Mitigación

Este riesgo se mitiga mediante la implementación de Consistencia Eventual con Apache Kafka. Dado que el catálogo, la geolocalización y las opiniones no impactan directamente los balances contables de la empresa ni los flujos de facturación, el negocio tolera esta brecha temporal corta.

Duplicidad de datos y mayor consumo de almacenamiento

Riesgo:

Al aplicar desnormalización por embedding en las colecciones de MongoDB para eliminar los costosos JOINS, la información estructural repetitiva eleva de forma drástica el consumo de almacenamiento en disco en comparación con un esquema relacional puro.

Estrategia de Mitigación

Se implementa una política estricta de Referenciación Ligera. Como se definió en el diseño de los JSONs, se embeben únicamente las dimensiones estáticas y las métricas precalculadas. Atributos pesados o transversales, como los diccionarios globales de traducción idiomática, no se duplican en MongoDB; en su lugar, se mantiene el string `product_category_name` como puente relacional, dejando que la capa de servicios o el API Gateway ensamble la traducción dinámicamente en memoria durante la capa de presentación.

Flujos de Sincronización

Para que la base de datos de MongoDB no quede aislada del estado operativo real de la plataforma, se define un modelo de Sincronización Circular Asíncrona Orientada a Eventos (Event-Driven). Este enfoque asegura que los motores cooperen sin bloquearse mutuamente:

Sincronización Transaccional a Consulta (Ida: PostgreSQL → MongoDB)

Cada vez que se ejecuta un cambio de estado importante en el Write Model (eje. una actualización administrativa de categorías), PostgreSQL confirma el cambio de manera ACID. Inmediatamente después del commit, el servicio correspondiente publica un evento en el Message Broker (Kafka). El microservicio del catálogo consume este mensaje en el trasfondo y realiza un update sobre el campo correspondiente en el documento NoSQL de MongoDB, actualizando el catálogo de forma transparente.

Sincronización de Interacción a Analítica (Vuelta: MongoDB → PostgreSQL):

Cuando un comprador genera datos que no alteran el flujo contable pero sí afectan al negocio como la inserción de una calificación de 5 estrellas en order_reviews, MongoDB persiste el documento de inmediato para asegurar baja latencia. En paralelo, un proceso de captura emite un evento del tipo ReviewSubmitted hacia Kafka. El motor consumidor de PostgreSQL toma este payload de forma asíncrona y actualiza la reputación acumulada del vendedor en la tabla relacional, protegiendo a la base de datos transaccional de la concurrencia masiva del front-end.